%1010_DDDD_EPPP_1111 << 16	
%1011_0000_0000_0000 << 16 %1011_DDDD_EPPP_0000 << 16	X_RFWORD_16P_2DAC8
%1011_0000_0000_0001 << 16 %1011_DDDD_EPPP_0001 << 16	X_RFLONG_32P_4DAC8
RDFAST → RGB → Pins / DACs	
%1011_0000_0000_0010 << 16 %1011_DDDD_EPPP_0010 << 16	X_RFBYTE_LUMA8
%1011_0000_0000_0011 << 16 %1011_DDDD_EPPP_0011 << 16	X_RFBYTE_RGBI8
%1011_0000_0000_0100 << 16 %1011_DDDD_EPPP_0100 << 16	X_RFBYTE_RGB8
%1011_0000_0000_0101 << 16 %1011_DDDD_EPPP_0101 << 16	X_RFWORD_RGB16
%1011_0000_0000_0110 << 16 %1011_DDDD_EPPP_0110 << 16	X_RFLONG_RGB24
Pins → DACs / WRFAST	
%1100_0000_0000_0000 << 16 %1100_DDDD_WPPP_PPPA << 16	X_1P_1DAC1_WFBYTE
%1101_0000_0000_0000 << 16 %1101_DDDD_WPPP_PP0A << 16	X_2P_2DAC1_WFBYTE
%1101_0000_0000_0010 << 16 %1101_DDDD_WPPP_PP1A << 16	X_2P_1DAC2_WFBYTE
%1110_0000_0000_0000 << 16 %1110_DDDD_WPPP_P00A << 16	X_4P_4DAC1_WFBYTE
%1110_0000_0000_0010 << 16 %1110_DDDD_WPPP_P01A << 16	X_4P_2DAC2_WFBYTE
%1110_0000_0000_0100 << 16 %1110_DDDD_WPPP_P10A << 16	X_4P_1DAC4_WFBYTE
%1110_0000_0000_0110 << 16 %1110_DDDD_WPPP_0110 << 16	X_8P_4DAC2_WFBYTE
%1110_0000_0000_0111 << 16 %1110_DDDD_WPPP_0111 << 16	X_8P_2DAC4_WFBYTE
%1110_0000_0000_1110 << 16 %1110_DDDD_WPPP_1110 << 16	X_8P_1DAC8_WFBYTE
%1110_0000_0000_1111 << 16 %1110_DDDD_WPPP_1111 << 16	X_16P_4DAC4_WFWORD
%1111_0000_0000_0000 << 16 %1111_DDDD_WPPP_0000 << 16	X_16P_2DAC8_WFWORD
%1111_0000_0000_0001 << 16 %1111_DDDD_WPPP_0001 << 16	X_32P_4DAC8_WFLONG
ADCs / Pins → DACs / WRFAST	
%1111_0000_0000_0010 << 16 %1111_DDDD_W000_0010 << 16	X_1ADC8_0P_1DAC8_WFBYTE
%1111_0000_0000_0011 << 16 %1111_DDDD_WPPP_0011 << 16	X_1ADC8_8P_2DAC8_WFWORD
%1111_0000_0000_0100 << 16 %1111_DDDD_W000_0100 << 16	X_2ADC8_0P_2DAC8_WFWORD
%1111_0000_0000_0101 << 16 %1111_DDDD_WPPP_0101 << 16	X_2ADC8_16P_4DAC8_WFLONG
%1111_0000_0000_0110 << 16 %1111_DDDD_W000_0110 << 16	X_4ADC8_0P_4DAC8_WFLONG
DDS / Goertzel	
%1111_0000_0000_0111 << 16 %1111_DDDD_0PPP_P111 << 16	X_DDS_GOERTZEL_SINC1
%1111_0000_1000_0111 << 16 %1111_DDDD_1PPP_P111 << 16	X_DDS_GOERTZEL_SINC2
Sub-Fields	
DAC Channel Outputs	
%xxxx_DDDD_xxxx_xxxx << 16 %0000_0000_0000_0000 << 16 %0000_0001_0000_0000 << 16 %0000_0010_0000_0000 << 16 %0000_0011_0000_0000 << 16 %0000_0100_0000_0000 << 16 %0000_0101_0000_0000 << 16 %0000_0110_0000_0000 << 16 %0000_0111_0000_0000 << 16 %0000_1000_0000_0000 << 16 %0000_1001_0000_0000 << 16	 X_DACS_OFF (default) X_DACS_0_0_0_0 X_DACS_X_X_0_0 X_DACS_0_0_X_X X_DACS_X_X_X_0 X_DACS_X_X_0_X X_DACS_X_0_X_X X_DACS_0_X_X_X X_DACS_0N0_0N0 X_DACS_X_X_0N0

%0000_1010_0000_0000 << 16 %0000_1011_0000_0000 << 16 %0000_1100_0000_0000 << 16 %0000_1101_0000_0000 << 16 %0000_1110_0000_0000 << 16 %0000_1111_0000_0000 << 16	X_DACS_0N0_X_X X_DACS_1_0_1_0 X_DACS_X_X_1_0 X_DACS_1_0_X_X X_DACS_1N1_0N0 X_DACS_3_2_1_0
Pin Output Control	
%xxxx_xxxx_Exxx_xxxx << 16 %0000_0000_0000_0000 << 16 %0000_0000_1000_0000 << 16	 X_PINS_OFF (default) X_PINS_ON
Write Control	
%xxxx_xxxx_Wxxx_xxxx << 16 %0000_0000_0000_0000 << 16 %0000_0000_1000_0000 << 16	 X_WRITE_OFF (default) X_WRITE_ON
Alternate Order for 1/2/4 bits	
%xxxx_xxxx_xxxx_xxxA << 16 %0000_0000_0000_0000 << 16 %0000_0000_0000_0001 << 16	 X_ALT_OFF (default) X_ALT_ON

Built-In Symbols for Events and Interrupt Sources

Symbol Value	Symbol Name
0	EVENT_INT / INT_OFF
1	EVENT_CT1
2	EVENT_CT2
3	EVENT_CT3
4	EVENT_SE1
5	EVENT_SE2
6	EVENT_SE3
7	EVENT_SE4
8	EVENT_PAT
9	EVENT_FBW
10	EVENT_XMT
11	EVENT_XFI
12	EVENT_XRO
13	EVENT_XRL
14	EVENT_ATN
15	EVENT_QMT

Built-In Symbols for COGINIT Usage

COGINIT Symbol Value	Symbol Name	Details
%00_0000	COGEXEC (default)	Use "COGEXEC + CogNumber" to start a cog in cogexec mode
%10_0000	HUBEXEC	Use "HUBEXEC + CogNumber" to start a cog in hubexec mode
%01_0000	COGEXEC_NEW	Starts an available cog in cogexec mode
%11_0000	HUBEXEC_NEW	Starts an available cog in hubexec mode
%01_0001	COGEXEC_NEW_PAIR	Starts an available eve/odd pair of cogs in cogexec mode, useful for LUT sharing
%11_0001	HUBEXEC_NEW_PAIR	Starts an available eve/odd pair of cogs in hubexec mode, useful for LUT sharing

Built-In Symbol for COGSPIN Usage

COGINIT Symbol Value	Symbol Name	Details
%01_0000	NEWCOG	Starts an available cog

Built-In Numeric Symbols

Symbol Value	Symbol Name	Details
$0000_0000	FALSE	Same as 0
$FFFF_FFFF	TRUE	Same as -1
$8000_0000	NEGX	Negative-extreme integer, -2_147_483_648 ($8000_0000)
$7FFF_FFFF	POSX	Positive-extreme integer, +2_147_483_647 ($7FFF_FFFF)
$4049_0FDB	PI	Single-precision floating-point value of Pi, 3.14159265

Command Line options for PNut.exe

Command	Action	ERROR.TXT file afterwards (file will contain one of these lines)
pnut	Start PNut.exe.	**okay**
pnut filename	Load *filename* (.spin2 extension is assumed, but not enforced).	**okay**
pnut filename -c	Load and compile *filename*, then exit.	**okay** <filename_path>**:**<line_number>**:error:**<error_message>
pnut filename -r	Load, compile, and run *filename*, then exit.	okay <filename_path>**:**<line_number>**:error:**<error_message> **serial_error**
pnut filename -rd	Load, compile, run, and debug *filename*, then exit when the Debug window gets closed.	**okay** <filename_path>**:**<line_number>**:error:**<error_message> **serial_error**
pnut filename -f	Load, compile, and program flash *filename*, then exit.	**okay** <filename_path>**:**<line_number>**:error:**<error_message> **serial_error**
pnut filename -fd	Load, compile, program flash, and debug *filename*, then exit when the Debug window gets closed.	**okay** <filename_path>**:**<line_number>**:error:**<error_message> **serial_error**
pnut -debug {CommPort} {BaudRate}	Open CommPort (default = 1) at BaudRate (default = 2_000_000) and present incoming DEBUG data and displays.	**okay** **serial_error**

Included Batch File to invoke PNut.exe and return status to STDOUT, STDERR, and ERRORLEVEL

PNUT_SHELL.BAT File	Batch File Line Descriptions
@echo off	Cancel echo to console.
set ERROR_FILE=error.txt	Set ERROR.TXT filename.
if exist %ERROR_FILE% del /q /f %ERROR_FILE%	If ERROR.TXT exists, delete it.
if exist %1 set GOOD_SRC=1	Check first parameter for a valid source file.
if exist %1.spin2 set GOOD_SRC=1	Check first parameter for a valid .spin2 source file.
if defined GOOD_SRC (	IF source file exists
pnut_v35L %1 %2 %3	...Invoke PNut with passed parameters. Example: pnut_shell filename -r
set pnuterror = %ERRORLEVEL%	...Capture ERRORLEVEL from PNut (0 = okay, 1 = error).
for /f "tokens=*" %%i in (%ERROR_FILE%) do echo %%i 1>&2	...Copy ERROR.TXT file to STDOUT and STDERR.
) else (	ELSE
set pnuterror=-1	...Set file-not-found error.
echo "Error: File NOT found - %1" 1>&2	...Return file-not-found error message to STDOUT and STDERR.
)	
exit %pnuterror%	Return ERRORLEVEL. Change to 'exit /b %pnuterror%' to maintain the console window.

Clock Setup

To establish the initial clock setup for your program, you can declare certain symbols which the compiler will look for to determine your setup. These symbols must be defined in one of the following combinations:

CON declarations (numbers are for example, can vary)	Effect	HUBSET %CC_SS **
`CON _clkfreq = 250_000_000` `_errfreq = 0`	Selects XI/XO-crystal-plus-PLL mode, assumes 20MHz crystal. The optimal PLL setting will be computed to achieve _clkfreq. Compilation fails if _clkfreq ± _errfreq is unachievable. *	`10_11`

`CON _xtlfreq = 12_000_000`
`_clkfreq = 148_500_000`
`_errfreq = 150_000`
	Selects XI/XO-crystal-plus-PLL mode, along with frequencies.
	The optimal PLL setting will be computed to achieve _clkfreq.
	Compilation fails if _clkfreq ± _errfreq is unachievable. *
		`1x_11`

`CON _xinfreq = 32_000_000`
`_clkfreq = 297_500_000`
`_errfreq = 100_000`
	Selects XI-input-plus-PLL mode, along with frequencies.
	The optimal PLL setting will be computed to achieve _clkfreq.
	Compilation fails if _clkfreq ± _errfreq is unachievable. *
		`01_10`

`CON _xtlfreq = 16_000_000`	Selects XI/XO-crystal mode and frequency.	`1x_10`
`CON _xinfreq = 100_000_000`	Selects XI-input mode and frequency.	`01_10`
`CON _rcslow`	Selects internal RCSLOW oscillator which runs at ~20KHz.	`00_01`
`CON _rcfast`	Selects internal RCFAST oscillator which runs at 20MHz+. This is the default mode, in case nothing is specified.	`00_00`

* The _errfreq declaration is optional, since _errfreq defaults to 1_000_000.

** If _xtlfreq >= 16_000_000 then x=0 for 15pF per XI/XO, else x=1 for 30pF per XI/XO.

During compilation, two constant symbols are defined by the compiler, whose values reflect the compiled clock setup:

Symbol	Description

`clkmode_`	The compiled clock mode, settable via HUBSET.
	● For Spin2 programs, HUBSET will be invoked with 'clkmode_' before your program starts, in order to set the compiled clock mode. The 'clkmode_' value will also be stored in the hub variable 'clkmode'.
	● For pure PASM programs, 'clkmode_' can be used to set the clock mode away from its initial RCFAST setting to any crystal/PLL compiled setting, as follows:
	`HUBSET ##clkmode_ & !3 'start crystal/PLL, stay in RCFAST`
	`WAITX ##20_000_000/100 'wait 10ms`
	`HUBSET ##clkmode_ 'switch to crystal/PLL`
	● The 'clkmode_' value may differ in each file of the application hierarchy. Files below the top-level file do not inherit the top-level file's value.

`clkfreq_`	The compiled clock frequency. ● For Spin2 programs, the 'clkfreq_' value will be stored in the hub variable 'clkfreq'. ● For pure PASM programs, 'clkfreq_' may be referenced only as a constant. ● The 'clkfreq_' value may differ in each file of the application hierarchy. Files below the top-level file do not inherit the top-level file's value.

For Spin2 programs, two hub variables are maintained which reflect the current clock setup:

Spin2 Variables	Description
`clkmode`	The current clock mode, located at LONG[$40]. Initialized with the 'clkmode_' value.
`clkfreq`	The current clock frequency, located at LONG[$44]. Initialized with the 'clkfreq_' value.

	● For Spin2 methods, these variables can be read and written as 'clkmode' and 'clkfreq'.
	Rather than write these variables directly, it's much safer to use:
	CLKSET(new_clkmode, new_clkfreq)
	This way, all other code sees a quick, parallel update to both 'clkmode' and 'clkfreq', and the clock mode transition is done safely, employing the prior values, in order to avoid a potential clock glitch.
	● For PASM code running under Spin2, these variables can be read and written as follows:
	`RDLONG x,#@clkmode 'read clkmode into x`
	`WRLONG x,#@clkmode 'write x to clkmode`
	`RDLONG x,#@clkfreq 'read clkfreq into x`
	`WRLONG x,#@clkfreq 'write x to clkfreq`
	`SETQ #2-1 'read clkmode and clkfreq into x and x+1`
	`RDLONG x,#@clkmode`
	`SETQ #2-1 'write x and x+1 to clkmode and clkfreq`
	`WRLONG x,#@clkmode`

For PASM-only programs, there is a special instruction named ASMCLK which will set the clock mode specified by the clock setup symbols. ASMCLK has no operands, but may be used with a conditional prefix. ASMCLK will assemble to one or six PASM instructions, depending upon the clock mode. This instruction is meant to be used once at the start of a PASM-only program, with the assumption that the RCFAST mode inherited from boot-up is currently selected:

CON declarations (numbers are for example, can vary)	HUBSET %CC_SS	ASMCLK assembles to:
CON _clkfreq = 250_000_000 _errfreq = 0	10_11	
CON _xtlfreq = 12_000_000 _clkfreq = 148_500_000 _errfreq = 150_000	1x_11	HUBSET ##clkmode_ & !%11 'start external clock, stay in RCFAST mode WAITX ##20_000_000/100 'allow 10ms for external clock to stabilize
CON _xinfreq = 32_000_000 _clkfreq = 297_500_000 _errfreq = 100_000	01_10	HUBSET ##clkmode_ 'switch to external clock mode
CON _xtlfreq = 16_000_000	1x_10	
CON _xinfreq = 100_000_000	01_10	
CON _rcslow	00_01	HUBSET #1 'switch to RCSLOW mode
CON _rcfast	00_00	HUBSET #0 'stay in RCFAST mode